

C-CLARKE INTERNATIONAL INSTITUTE OF DIGITAL SCIENCES

Database Scripts

Real, runnable schema DDL generated directly from the live SQLAlchemy models.

Program	Diploma in Advanced AI & Software Engineering
Batch	25/26
Project	Smart Hire — Service Marketplace
Team	Nilan Iddawela, Janaka Eranda, Dasun Anjana, Damith Lasantha, Matheesha Milan
Date	21 July 2026

1. How the Schema Is Managed

The application defines its schema as SQLAlchemy ORM models (`backend/app/models/`) and creates any missing tables automatically on startup via `Base.metadata.create_all(engine)` in `backend/app/main.py` . The statements below are the real, exact DDL generated from those models for the current live schema (16 tables) — they can be run directly against a fresh PostgreSQL database to recreate the structure.

These are the actual current model definitions, not a historical or aspirational schema. Enum types (`user_role` , `account_status` , `booking_status` , `verification_status`) are created automatically by SQLAlchemy/PostgreSQL when the tables are created.

2. Regenerating This Script

```
cd backend && source venv/bin/activate && python -c "
from app.core.database import Base
from app.models import *
from sqlalchemy.schema import CreateTable
from sqlalchemy.dialects import postgresql
for t in Base.metadata.sorted_tables:
    print(CreateTable(t).compile(dialect=postgresql.dialect()))
"
```

3. Full Schema DDL (16 tables)

keyword **data type** **table name** **number** punctuation

service_categories

```
CREATE TABLE service_categories (
    id SERIAL NOT NULL,
    name VARCHAR(100) NOT NULL,
    description TEXT,
    icon VARCHAR(20) NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE NOT NULL,
    PRIMARY KEY (id)
);
```

users

```
CREATE TABLE users (  
  id SERIAL NOT NULL,  
  email VARCHAR(255) NOT NULL,  
  password_hash VARCHAR(255) NOT NULL,  
  full_name VARCHAR(150) NOT NULL,  
  phone VARCHAR(20),  
  role user_role NOT NULL,  
  status account_status NOT NULL,  
  email_verified BOOLEAN NOT NULL,  
  created_at TIMESTAMP WITH TIME ZONE NOT NULL,  
  updated_at TIMESTAMP WITH TIME ZONE NOT NULL,  
  PRIMARY KEY (id)  
);
```

ai_conversations

```
CREATE TABLE ai_conversations (  
  id SERIAL NOT NULL,  
  user_id INTEGER NOT NULL,  
  title VARCHAR(255) NOT NULL,  
  created_at TIMESTAMP WITH TIME ZONE NOT NULL,  
  updated_at TIMESTAMP WITH TIME ZONE NOT NULL,  
  PRIMARY KEY (id),  
  FOREIGN KEY(user_id) REFERENCES users (id) ON DELETE CASCADE  
);
```

provider_profiles

```
CREATE TABLE provider_profiles (  
  user_id INTEGER NOT NULL,  
  bio TEXT,  
  years_experience INTEGER NOT NULL,  
  verification_status verification_status NOT NULL,  
  avg_rating NUMERIC(3, 2) NOT NULL,  
  total_reviews INTEGER NOT NULL,  
  created_at TIMESTAMP WITH TIME ZONE NOT NULL,  
  updated_at TIMESTAMP WITH TIME ZONE NOT NULL,  
  PRIMARY KEY (user_id),  
  CONSTRAINT ck_provider_profiles_years_experience_non_negative CHECK (years_experience >= 0),  
  CONSTRAINT ck_provider_profiles_avg_rating_range CHECK (avg_rating >= 0 AND avg_rating <= 5),  
  CONSTRAINT ck_provider_profiles_total_reviews_non_negative CHECK (total_reviews >= 0),  
  FOREIGN KEY(user_id) REFERENCES users (id) ON DELETE CASCADE  
);
```

services

```
CREATE TABLE services (  
  id SERIAL NOT NULL,  
  provider_id INTEGER NOT NULL,  
  category_id INTEGER NOT NULL,  
  title VARCHAR(160) NOT NULL,  
  description TEXT NOT NULL,  
  price NUMERIC(12, 2) NOT NULL,  
  city VARCHAR(100) NOT NULL,  
  duration VARCHAR(60) NOT NULL,  
  status service_status NOT NULL,  
  created_at TIMESTAMP WITH TIME ZONE NOT NULL,  
  updated_at TIMESTAMP WITH TIME ZONE NOT NULL,  
  PRIMARY KEY (id),  
  FOREIGN KEY(provider_id) REFERENCES users (id) ON DELETE CASCADE,  
  FOREIGN KEY(category_id) REFERENCES service_categories (id)  
);
```

ai_messages

```
CREATE TABLE ai_messages (  
  id SERIAL NOT NULL,  
  conversation_id INTEGER NOT NULL,  
  role VARCHAR(20) NOT NULL,  
  message TEXT NOT NULL,  
  created_at TIMESTAMP WITH TIME ZONE NOT NULL,  
  PRIMARY KEY (id),  
  FOREIGN KEY(conversation_id) REFERENCES ai_conversations (id) ON DELETE CASCADE  
);
```

bookings

```
CREATE TABLE bookings (  
  id SERIAL NOT NULL,  
  customer_id INTEGER NOT NULL,  
  provider_id INTEGER NOT NULL,  
  service_id INTEGER NOT NULL,  
  booking_date TIMESTAMP WITH TIME ZONE NOT NULL,  
  status booking_status NOT NULL,  
  notes TEXT,  
  created_at TIMESTAMP WITH TIME ZONE NOT NULL,  
  updated_at TIMESTAMP WITH TIME ZONE NOT NULL,  
  PRIMARY KEY (id),  
  FOREIGN KEY(customer_id) REFERENCES users (id) ON DELETE CASCADE,  
  FOREIGN KEY(provider_id) REFERENCES users (id) ON DELETE CASCADE,  
  FOREIGN KEY(service_id) REFERENCES services (id) ON DELETE CASCADE  
);
```

favorites

```
CREATE TABLE favorites (  
  id SERIAL NOT NULL,  
  user_id INTEGER NOT NULL,  
  service_id INTEGER NOT NULL,  
  created_at TIMESTAMP WITH TIME ZONE NOT NULL,  
  PRIMARY KEY (id),  
  CONSTRAINT uq_favorites_user_service UNIQUE (user_id, service_id),  
  FOREIGN KEY(user_id) REFERENCES users (id) ON DELETE CASCADE,  
  FOREIGN KEY(service_id) REFERENCES services (id) ON DELETE CASCADE  
);
```

review_summaries

```
CREATE TABLE review_summaries (  
  id SERIAL NOT NULL,  
  service_id INTEGER NOT NULL,  
  summary TEXT NOT NULL,  
  PRIMARY KEY (id),  
  UNIQUE (service_id),  
  FOREIGN KEY(service_id) REFERENCES services (id) ON DELETE CASCADE  
);
```

service_areas

```
CREATE TABLE service_areas (  
    id SERIAL NOT NULL,  
    provider_id INTEGER NOT NULL,  
    service_id INTEGER,  
    district VARCHAR(100) NOT NULL,  
    city VARCHAR(100) NOT NULL,  
    radius_km INTEGER NOT NULL,  
    PRIMARY KEY (id),  
    FOREIGN KEY(provider_id) REFERENCES users (id) ON DELETE CASCADE,  
    FOREIGN KEY(service_id) REFERENCES services (id) ON DELETE CASCADE  
);
```

service_embeddings

```
CREATE TABLE service_embeddings (  
    id SERIAL NOT NULL,  
    service_id INTEGER NOT NULL,  
    embedding TEXT NOT NULL,  
    PRIMARY KEY (id),  
    UNIQUE (service_id),  
    FOREIGN KEY(service_id) REFERENCES services (id) ON DELETE CASCADE  
);
```

service_images

```
CREATE TABLE service_images (  
    id SERIAL NOT NULL,  
    service_id INTEGER NOT NULL,  
    image_url TEXT NOT NULL,  
    PRIMARY KEY (id),  
    FOREIGN KEY(service_id) REFERENCES services (id) ON DELETE CASCADE  
);
```

messages

```
CREATE TABLE messages (  
    id SERIAL NOT NULL,  
    booking_id INTEGER NOT NULL,  
    sender_id INTEGER NOT NULL,  
    body TEXT NOT NULL,  
    is_read BOOLEAN NOT NULL,  
    created_at TIMESTAMP WITH TIME ZONE NOT NULL,  
    PRIMARY KEY (id),  
    FOREIGN KEY(booking_id) REFERENCES bookings (id) ON DELETE CASCADE,  
    FOREIGN KEY(sender_id) REFERENCES users (id) ON DELETE CASCADE  
);
```

notifications

```
CREATE TABLE notifications (  
    id SERIAL NOT NULL,  
    user_id INTEGER NOT NULL,  
    title VARCHAR(150) NOT NULL,  
    message TEXT NOT NULL,  
    is_read BOOLEAN NOT NULL,  
    booking_id INTEGER,  
    created_at TIMESTAMP WITH TIME ZONE NOT NULL,  
    PRIMARY KEY (id),  
    FOREIGN KEY(user_id) REFERENCES users (id) ON DELETE CASCADE,  
    FOREIGN KEY(booking_id) REFERENCES bookings (id) ON DELETE CASCADE  
);
```

payments

```
CREATE TABLE payments (  
    id SERIAL NOT NULL,  
    booking_id INTEGER NOT NULL,  
    customer_id INTEGER NOT NULL,  
    amount FLOAT NOT NULL,  
    status VARCHAR,  
    payment_method VARCHAR,  
    transaction_id VARCHAR,  
    created_at TIMESTAMP WITHOUT TIME ZONE,  
    updated_at TIMESTAMP WITHOUT TIME ZONE,  
    PRIMARY KEY (id),  
    FOREIGN KEY(booking_id) REFERENCES bookings (id),  
    FOREIGN KEY(customer_id) REFERENCES users (id),  
    UNIQUE (transaction_id)  
);
```

reviews

```
CREATE TABLE reviews (  
    id SERIAL NOT NULL,  
    booking_id INTEGER NOT NULL,  
    customer_id INTEGER NOT NULL,  
    provider_id INTEGER NOT NULL,  
    service_id INTEGER NOT NULL,  
    rating INTEGER NOT NULL,  
    comment TEXT,  
    created_at TIMESTAMP WITH TIME ZONE,  
    updated_at TIMESTAMP WITH TIME ZONE,  
    PRIMARY KEY (id),  
    UNIQUE (booking_id),  
    FOREIGN KEY(booking_id) REFERENCES bookings (id),  
    FOREIGN KEY(customer_id) REFERENCES users (id),  
    FOREIGN KEY(provider_id) REFERENCES users (id),  
    FOREIGN KEY(service_id) REFERENCES services (id)  
);
```